PROJECT_STRUCTURE_JUMPAVERSE

Project architecture, UML diagrams, runtime logic, and code snippets

PROJECT_STRUCTURE_JUMPAVERSE.pdf

This document explains the structure of the Jumpa Verse SDL2 project: files, classes, dependencies, runtime flow, object data, and the main logic used by the game.

1. Project Purpose

Jumpa Verse is a C++ SDL2 side-scrolling game prototype. The project uses SDL2 for window creation, input, timing, and rendering; SDL_image for texture loading; SDL_ttf for menu text; and SDL2_gfx for triangular spike traps.

- The player sees a fixed SDL window.
- Ground and traps move left to create a camera illusion.
- New platforms are generated randomly after the latest platform.
- Old ground and traps are removed so vectors do not grow forever.
- The menu is rendered with SDL_ttf text textures and SDL rectangle drawing.

2. File Architecture

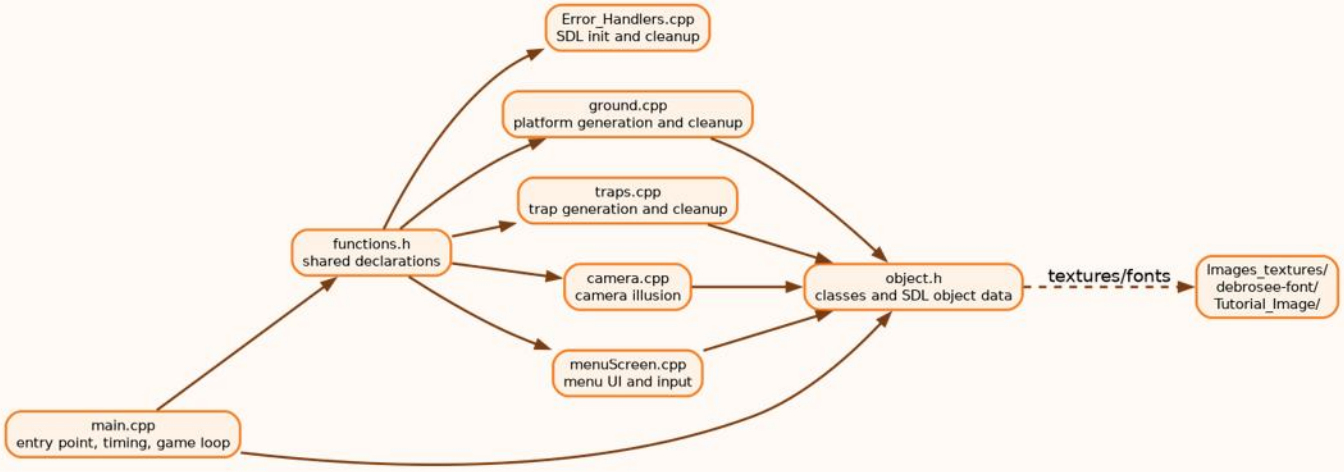

Figure 1: File-level project architecture

- main.cpp is the control center. It initializes the game, runs the menu, updates the world, renders frames, limits FPS, and shuts down.
- functions.h is the common interface. It declares the functions used across multiple source files.
- object.h stores the core object structures and classes used by rendering and logic.
- Error_Handlers.cpp initializes SDL, SDL_image, SDL_ttf, window, and renderer.

- ground.cpp contains platform creation, procedural ground generation, and ground cleanup.
- traps.cpp contains spike creation and trap cleanup.
- camera.cpp moves world objects left to create side-scrolling movement.
- menuScreen.cpp builds and renders the Play/Exit menu.

3. UML Class Structure

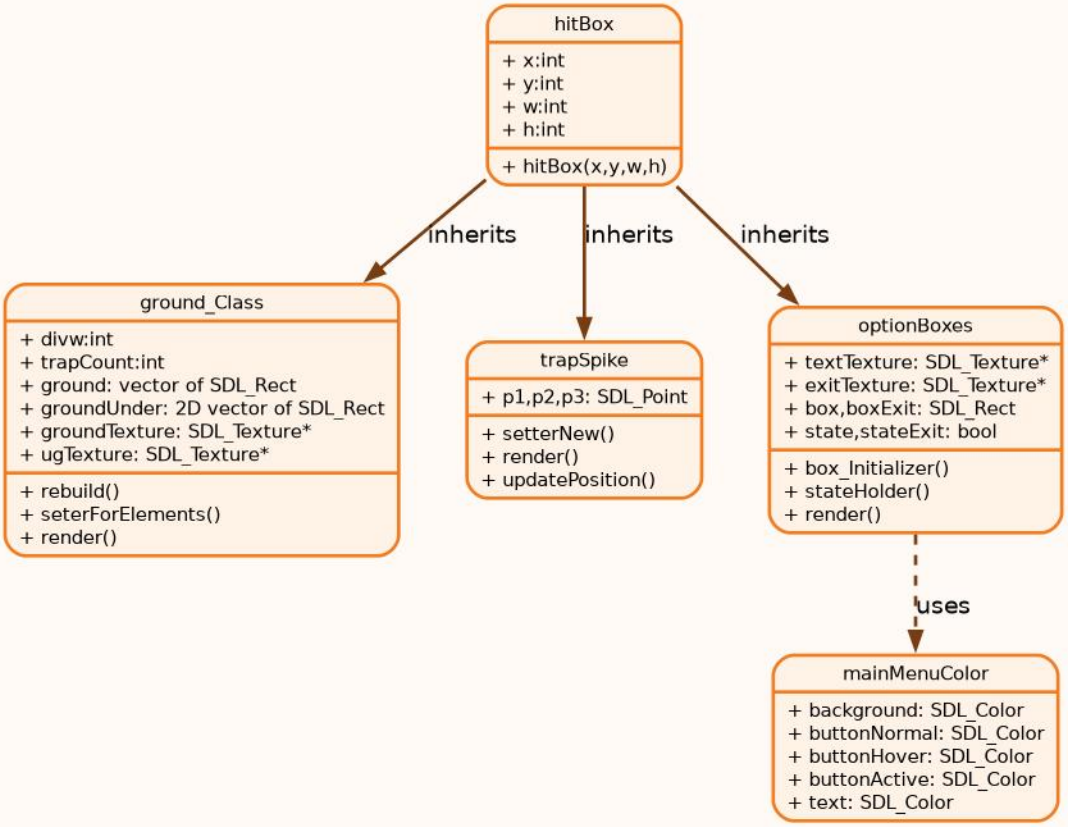

Figure 2: UML class structure of Jumpa Verse

The shared idea in the project is that visual objects need a position and size. The hitBox class provides x, y, w, and h. ground_Class, trapSpike, and optionBoxes inherit those fields and then add their own rendering data.

3.1 hitBox

hitBox is the simplest structure in the project. It stores the rectangle-like area of an object. For beginners, it is useful to think of hitBox as the address and size of something on the SDL screen.

```
class hitBox
{
public:
int x, y, w, h;

hitBox(int x, int y, int w, int h)
{
this->x = x;
this->y = y;
```

```
this->w = w;
this->h = h;
}
};
```

3.2 ground_Class

ground_Class represents one platform. It does not draw one huge rectangle. Instead, it divides the platform into 50x50 tile rectangles. The top row uses groundTexture and the lower body uses ugTexture.

```
void rebuild()
{
divw = w / perboxwidth;
int rows = h / perboxheight;

ground.clear();
groundUnder.clear();
ground.resize(divw);
groundUnder.resize(divw);
}
```

Logic used: rebuild() recalculates tile positions whenever x, y, w, or h changes. This matters because camera movement changes x positions every frame.

3.3 trapSpike

trapSpike stores three triangle points. SDL2 itself does not include filled triangle drawing, so the project uses SDL2_gfx functions filledTrigonRGBA and trigonRGBA.

```
trapSpike(int x,int y,int w,int h):hitBox(x,y,w,h)
{
p1 = {x, y};
p2 = {x + 50, y};
p3 = {x + 25, y - 50};
}
```

Logic used: p1 and p2 sit on the ground line. p3 is above them, so the triangle points upward. Because SDL y coordinates increase downward, y - 50 moves the top point upward.

3.4 optionBoxes

optionBoxes renders Play and Exit buttons. It turns text into SDL textures using SDL_ttf, then draws those textures centered inside button rectangles.

```
SDL_Surface* surface1 = TTF_RenderText_Blended(font, text.c_str(), color);
textTexture = SDL_CreateTextureFromSurface(renderer, surface1);
SDL_FreeSurface(surface1);
```

Logic used: text starts as font data, becomes an SDL_Surface, then becomes an SDL_Texture. The surface is temporary and must be freed after the texture is created.

4. Runtime Flow

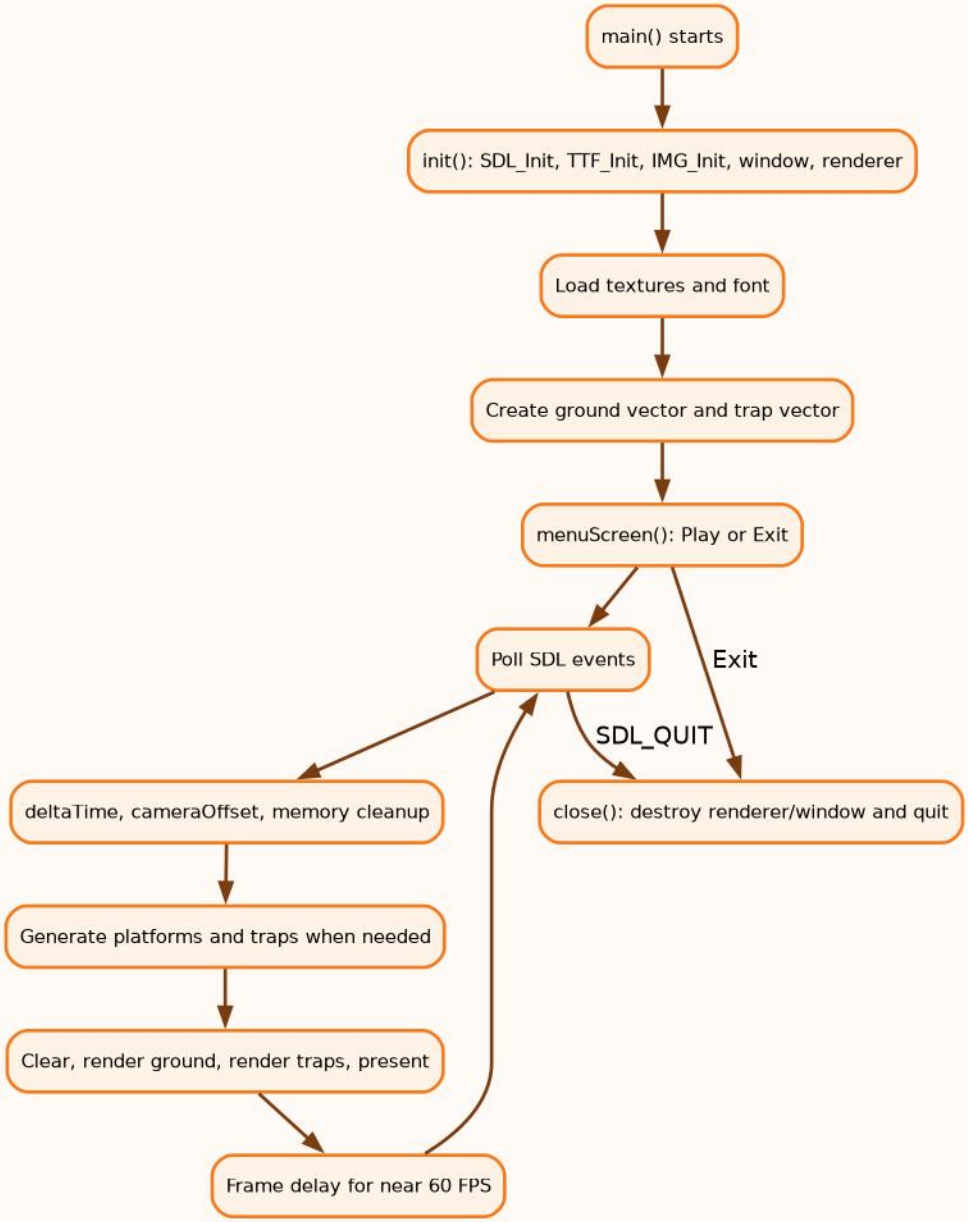

Figure 3: Runtime flow from main() to cleanup

The program lifecycle is simple: initialize, load assets, create initial world state, show menu, enter the game loop, render frames, and clean up resources when the user exits.

```
if (!init(window, renderer))
{
std::cerr << "FAILD TO START" << std::endl;
return -1;
}

SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", 100);
```

5. Main Game Loop Logic

The game loop is the most important structure in the project. It repeats until quit becomes true. Each loop processes input, updates game state, renders the frame, and limits frame speed.

```
while (!quit)
{
Uint32 frameStart = SDL_GetTicks();
float deltaTime = (currentTime - lastTime) / 1000.0f;

while (SDL_PollEvent(&eventManager) != 0)
{
if (eventManager.type == SDL_QUIT)
quit = true;
}

SDL_RenderClear(renderer);
// render ground and traps
SDL_RenderPresent(renderer);
}
```

- Input stage: SDL_PollEvent checks close, keyboard, and mouse events.
- Update stage: deltaTime and cameraOffset decide how far the world moves.
- Generation stage: new ground and traps are added when the game needs more world.
- Cleanup stage: old objects are removed from vectors.
- Render stage: the screen is cleared, objects are drawn, and the frame is presented.

6. Data Flow

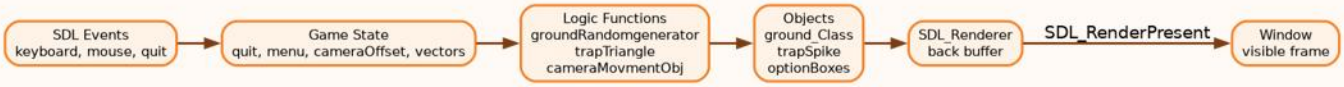

Figure 4: Data flow from input to visible frame

SDL events change state variables such as quit, menu, and button selection. Logic functions use that state to modify objects. Objects render themselves through SDL_Renderer, and SDL_RenderPresent shows the completed frame.

7. Procedural Ground Logic

The ground generator uses the last platform as a reference. The next platform is placed after the previous platform, with a random gap, random y value, random width, and random height.

```
std::uniform_int_distribution<> distx(
prevGround.back().x + prevGround.back().w + 120,
prevGround.back().x + prevGround.back().w + 200
);

std::uniform_int_distribution<> disty(
prevGround.back().y - 100,
prevGround.back().y + 600
);
```

Reasoning: using prevGround.back() keeps the world connected. If every platform used a completely random x position, platforms could overlap, appear behind the player, or leave impossible gaps.

8. Trap Logic

Traps are generated with chance. A platform can receive a trap only if the random chance succeeds and the platform has not reached its trap limit.

```
if (chances(gen) < 5 && ground[i].trapCount < 4)
{
ground[i].trapCount++;
trap.emplace_back(distp(gen), ground[i].y, ground[i].w, ground[i].h);
break;
}
```

Reasoning: random chance makes gameplay less predictable. trapCount prevents one platform from becoming overloaded with traps.

9. Camera Illusion Logic

SDL2 does not provide a built-in 2D game camera. Jumpa Verse creates the camera effect by moving world objects left while the window stays fixed.

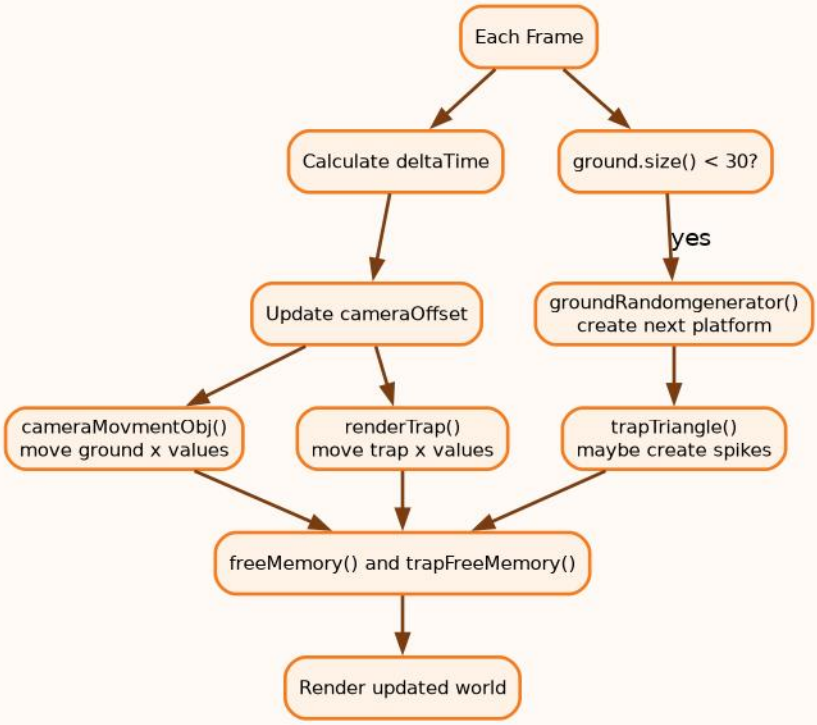

Figure 5: Camera, generation, cleanup, and render logic map

```
ground[i].seterForElements(
ground[i].x + (int)cameraOffset,
ground[i].y,
ground[i].w,
ground[i].h
);
```

Reasoning: when every ground object shifts left, the player feels as if the world is scrolling. The renderer is not moving; the object coordinates are moving.

9.1 Camera Logic Step By Step

- main.cpp starts with cammera_offSet = 0.5f. This value is used as the movement amount added to object x positions.
- Every frame calculates deltaTime from SDL_GetTicks. This tells the program how much real time passed since the previous frame.
- The code decreases cammera_offSet while it is greater than -9.0f. This makes the offset negative, so world objects move left.
- cameraMovmentObj() loops through every ground_Class object and updates its x value.
- seterForElements() changes the platform x/y/w/h and calls rebuild(), so all SDL_Rect tile positions are recalculated.
- renderTrap() loops through every trapSpike and calls setterNew(), which recalculates p1, p2, and p3 for the triangle.
- The result is a scrolling-world effect: the SDL window remains still, but platforms and traps slide left.

```
if (cammera_offSet > -9.0f)
{
cammera_offSet -= speed * deltaTime;
}

cameraMovmentObj(ground, cammera_offSet, trap);
renderTrap(renderer, trap, cammera_offSet);
```

9.2 Camera Logic Possible Errors

- If cameraOffset is too large, platforms jump instead of scrolling smoothly.
- If cameraOffset becomes 0 after casting to int, movement can look stuck for some frames.
- If ground moves but traps do not, traps will visually separate from the platforms.
- If rebuild() is not called after changing platform x, the platform hitBox and its SDL_Rect tiles can disagree.
- If deltaTime becomes very large after lag, the world can suddenly skip forward. The project reduces this by clamping deltaTime to 0.05f.
- If old objects are not erased, vectors grow and rendering becomes slower over time.

10. Generation Logic

Generation logic is the part of the project that keeps the world alive. Instead of designing every platform manually, the game creates new platforms and traps while the player moves forward.

10.1 Ground Generation Logic

ground_generator() creates the first platform. groundRandomgenerator() creates later platforms based on the last platform in the vector.

```
if (ground.size() < 30)
{
temp = groundRandomgenerator(renderer, groundTex, ugTex, ground, trap);
```

```
ground.push_back(temp);
}
```

- The vector ground stores all active platforms.
- When ground.size() is less than 30, the game creates another platform.
- The new platform x starts after prevGround.back().x + prevGround.back().w, so it appears after the latest platform.
- The y value is selected near the previous platform y value, then checked so it stays inside a playable range.
- The width and height are random, so platforms vary in size.
- The new ground_Class receives shared ground and underground textures instead of loading new images.

10.2 Trap Generation Logic

trapTriangle() is called while generating ground. It uses random chance and a trap limit per platform.

```
std::uniform_int_distribution<> chances(0, 100);

if (chances(gen) < 5 && ground[i].trapCount < 4)
{
ground[i].trapCount++;
trap.emplace_back(distp(gen), ground[i].y, ground[i].w, ground[i].h);
break;
}
```

- distp chooses a possible trap x position inside the platform width.
- chances(gen) < 5 means the trap appears only when the random number is very small.
- ground[i].trapCount < 4 prevents too many traps on one platform.
- trap.emplace_back(...) constructs the trap directly inside the trap vector.
- The trap y value uses ground[i].y, so the triangle sits on top of the platform.

10.3 Generation Logic Possible Errors

- If prevGround is empty, prevGround.back() is invalid and can crash the program.
- If random y is not limited, platforms can appear outside the playable screen area.
- If x gap is too large, jumps may become impossible.
- If x gap is too small, platforms overlap too much.
- If trap chance is too high, gameplay becomes unfair.
- If trap chance is too low, traps may rarely appear.
- If old platforms are not removed, the vector keeps growing and the game slows down.

11. Error Logic

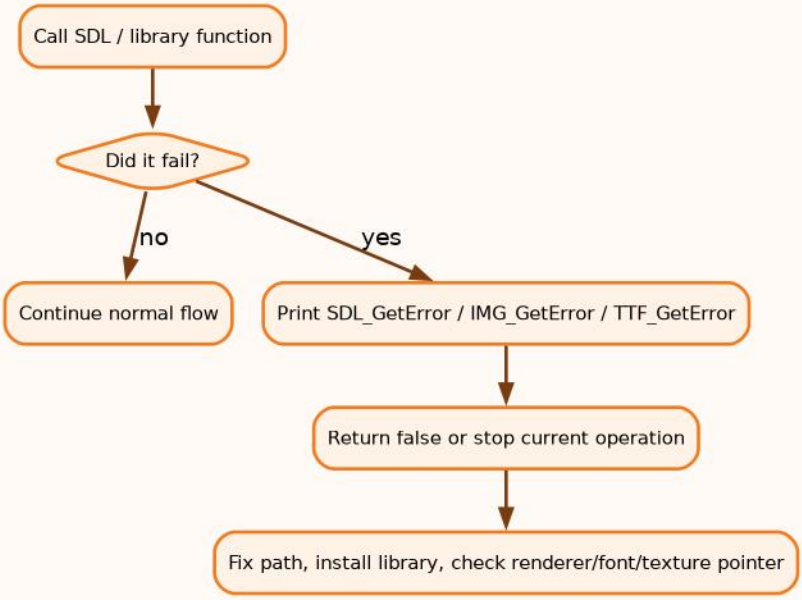

Figure 6: Error-checking flow used by SDL project code

Error logic means checking every risky SDL operation before the project continues. SDL functions often return nullptr, false-like values, or negative numbers when something fails.

11.1 Initialization Error Logic

```
if (SDL_Init(SDL_INIT_VIDEO) < 0)
{
std::cerr << "SDL couldnt be initialized! Error Type: "
<< SDL_GetError() << std::endl;
return false;
}
```

- SDL_Init can fail if the video subsystem cannot start.
- TTF_Init can fail if SDL_ttf is missing or cannot initialize.
- IMG_Init can fail if JPEG image support is missing.
- SDL_CreateWindow can return nullptr if the window cannot be created.
- SDL_CreateRenderer can return nullptr if accelerated rendering is unavailable.

11.2 Asset Error Logic

Assets are external files, so paths can be wrong even when the code is correct. Textures and fonts should always be checked after loading.

```
SDL_Texture* groundTex = IMG_LoadTexture(renderer, "Images_textures/ground.jpeg");
TTF_Font* font = TTF_OpenFont("debrosee-font/Debrosee-ALPnL.ttf", 100);

if (!groundTex)
std::cerr << IMG_GetError() << std::endl;
if (!font)
std::cerr << TTF_GetError() << std::endl;
```

- If IMG_LoadTexture returns nullptr, the file path may be wrong or SDL_image may not support that format.

- If TTF_OpenFont returns nullptr, the font file may be missing or the working directory may be wrong.
- If SDL_CreateTextureFromSurface returns nullptr, the renderer or surface may be invalid.
- If the game is run from outside the project root, relative asset paths can fail.

11.3 Runtime Error Logic

- Before rendering a texture, check that the SDL_Texture pointer is not nullptr.
- Before using vector.back(), make sure the vector is not empty.
- Before erasing vector elements in a loop, update the index carefully so elements are not skipped.
- Before drawing rectangles, make sure width and height are positive.
- Before using a font in menuScreen(), check that font is not nullptr.

12. Rendering Order

Rendering order matters. Background must be cleared first, then ground is drawn, then traps are drawn. If the order is wrong, objects may cover each other incorrectly.

```
SDL_SetRenderDrawColor(renderer, 135, 206, 235, 255);
SDL_RenderClear(renderer);

for (int j = 0; j < ground.size(); j++)
ground[j].render(renderer);

for (int j = 0; j < trap.size(); j++)
trap[j].render(renderer);

SDL_RenderPresent(renderer);
```

13. Menu Logic

The menu uses mouse position, mouse clicks, keyboard navigation, and SDL_ttf text textures. It returns true when Play or Exit is selected. The global exitb flag tells main.cpp when Exit was selected.

```
if (mx >= playButton.x && mx <= playButton.x + playButton.w &&
my >= playButton.y && my <= playButton.y + playButton.h)
{
return true;
}
```

Reasoning: button click detection is rectangle collision. If the mouse x/y is inside the button rectangle, the click belongs to that button.

14. Memory Structure

- ground is a vector of ground_Class objects.
- trap is a vector of trapSpike objects.
- Textures are loaded once and shared with many ground objects.
- Text textures are owned by optionBoxes and destroyed in its destructor.

- Old ground is erased when it moves too far across the screen.
- Old traps are erased when trap vector size grows beyond the chosen limit.

```
void trapFreeMemory(std::vector<trapSpike>& trap)
{
if (trap.size() > 20)
{
trap.erase(trap.begin());
}
}
```

15. Improvement Notes

- Add a player class to separate player state from the world logic.
- Close the opened TTF_Font and call TTF_Quit during cleanup.
- Destroy groundTex and ugTex after the loop ends.
- Use constants for window size, tile size, movement speed, and colors.
- Split the main loop into input(), update(), render(), and cleanup() functions when the project grows.
- Add collision detection between player hitBox, ground_Class rectangles, and trapSpike triangles.